NAME:U.Prabhavathi **COURSE CODE:DSA0107**

REG NO:192472067 **COURSE NAME:C++**

UNIT II
FUNCTIONS AND CLASSES & OBJECTS IN C++

Solved Assignment – Debugging, Corrected Programs and Pseudo-Code

Prepared according to the supplied UNIT II question bank. The five debugging programs are corrected, compiled with g++ (C++17), and executed. Output screenshots are included after each program. Section B contains structured pseudo-code solutions.

SECTION A – DEBUGGING QUESTIONS

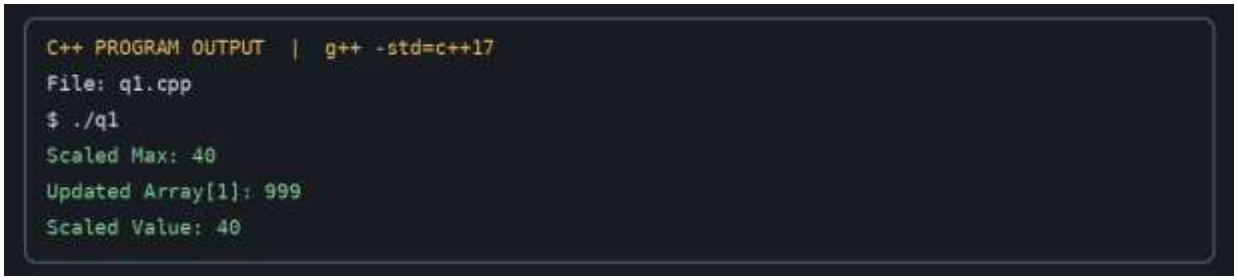
1. Dangling Reference in Return-by-Reference

- Defect: getMaximumScaled() returns a reference to the local variable result. The variable is destroyed when the function ends, so the reference becomes dangling and reading it causes undefined behavior.
- Correction: return the calculated integer by value. getArrayElement() can safely return a reference because it refers to an element in the array owned by the caller, whose lifetime continues in main().

Corrected C++ Program:

```
#include <iostream>
int getMaximumScaled(int a, int b, int scaleFactor) {
   int maxVal = (a > b) ? a : b;
   return maxVal * scaleFactor; // return by value
}
int& getArrayElement(int arr[], int index) {
   return arr[index]; // safe: refers to caller-owned array element
}
int main() {
   int x = 10, y = 20;
   int scaled = getMaximumScaled(x, y, 2);
   std::cout << "Scaled Max: " << scaled << '\n';
   int data[3] = {100, 200, 300};
   getArrayElement(data, 1) = 999;
   std::cout << "Updated Array[1]: " << data[1] << '\n';
   std::cout << "Scaled Value: " << scaled << '\n';
   return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

2. Static Data Member Linkage and Function Scope

- Defects: totalBankReserves is declared in the class but needs one out-of-class definition to provide storage. Also, static displayAudit() cannot access accHolder because accHolder belongs to an individual object and there is no this pointer in a static member function.
- Correction: define double BankAccount::totalBankReserves = 0.0; outside the class. Keep the static audit method for total reserves and use a non-static displayAccount() method for each account holder.

Corrected C++ Program:

```
#include <iostream>
#include <string>
class BankAccount {
private:
   std::string accHolder;
   double balance;
   static double totalBankReserves;
public:
   BankAccount(std::string name, double deposit)
      : accHolder(name), balance(deposit) {
      totalBankReserves += deposit;
   }
   static void displayAudit() {
      std::cout << "Total Reserves in Vault: $" << totalBankReserves << '\n';
   }
   void displayAccount() const {
      std::cout << "Account Holder: " << accHolder
            << ", Balance: $" << balance << '\n';
   }
};
double BankAccount::totalBankReserves = 0.0;
int main() {
   BankAccount acc1("Alice", 5000.0);
   BankAccount acc2("Bob", 3000.0);
   BankAccount::displayAudit();
```

```
    acc1.displayAccount();
    acc2.displayAccount();
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

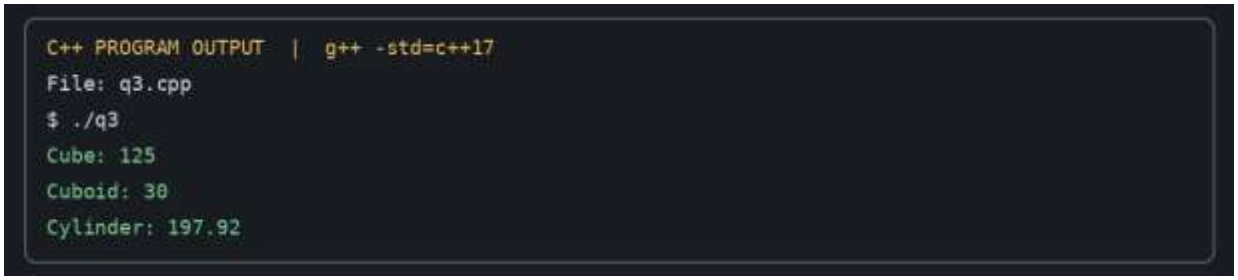
3. Function Overload Ambiguity via Default Arguments

- Defect: computeVolume(5.0) can match both the three-parameter overload with default values and the one-parameter cube overload. This makes the call ambiguous.
- Correction: use distinct descriptive names—computeCuboidVolume(), computeCylinderVolume(), and computeCubeVolume(). This keeps all three calculations available without overload ambiguity.

Corrected C++ Program:

```
#include <iostream>
double computeCuboidVolume(double length, double width = 1.0,
                           double height = 1.0) {
    return length * width * height;
}
double computeCylinderVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}
double computeCubeVolume(double side) {
    return side * side * side;
}
int main() {
    std::cout << "Cube: " << computeCubeVolume(5.0) << '\n';
    std::cout << "Cuboid: " << computeCuboidVolume(5.0, 2.0, 3.0) << '\n';
    std::cout << "Cylinder: " << computeCylinderVolume(3.0, 7.0) << '\n';
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

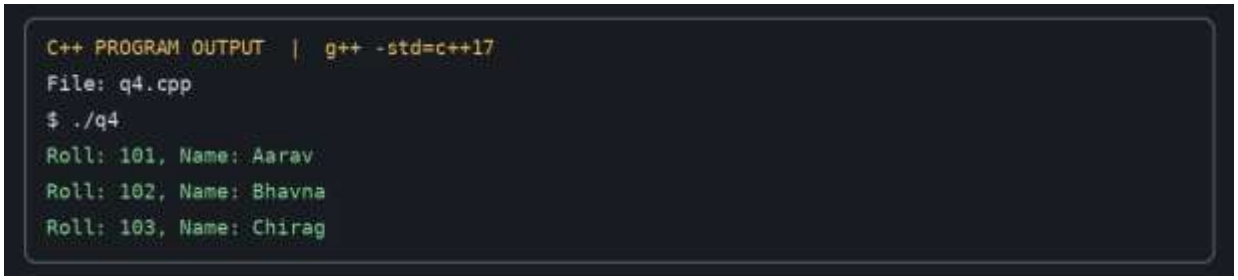
4. Array of Objects Missing Default Constructor

- Defect: Student batch[3] requires three Student objects to be default-constructed. Once a parameterized constructor is declared, the compiler does not automatically provide a default constructor.
- Correction: add an explicit zero-argument constructor. The array can then be created and its elements assigned parameterized Student objects.

Corrected C++ Program:

```
#include <iostream>
#include <string>
class Student {
private:
   int rollNumber;
   std::string name;
public:
   Student() : rollNumber(0), name("Not assigned") {}
   Student(int r, std::string n) : rollNumber(r), name(n) {}
   void display() const {
      std::cout << "Roll: " << rollNumber << ", Name: " << name << '\n';
   }
};
int main() {
   Student batch[3];
   batch[0] = Student(101, "Aarav");
   batch[1] = Student(102, "Bhavna");
   batch[2] = Student(103, "Chirag");
   for (int i = 0; i < 3; ++i) batch[i].display();
   return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

5. Friend Function Declaration and Scope

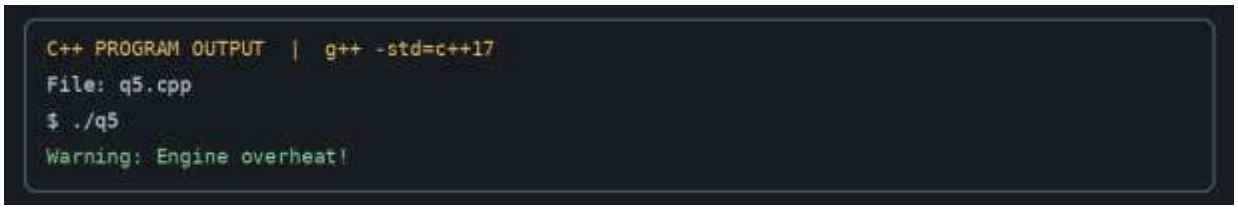
- Defects: TelemetryAuditor must be forward-declared before it is used in Engine's friend declaration. Inside the friend function, private data must be accessed through the passed objects (a.alertThreshold). A friend function is not a class member and has no this pointer.
- Correction: forward-declare TelemetryAuditor, declare the same friend function in both classes, qualify private-member access through e and a, and call inspectDiagnostics(v8, audit) like an ordinary function.

Corrected C++ Program:

```
#include <iostream>
class TelemetryAuditor;
class Engine {
private:
   int temperature, rpm;
public:
   Engine(int t, int r) : temperature(t), rpm(r) {}
   friend void inspectDiagnostics(const Engine&, const TelemetryAuditor&);
};
class TelemetryAuditor {
private:
   int alertThreshold;
public:
   TelemetryAuditor(int threshold) : alertThreshold(threshold) {}
   friend void inspectDiagnostics(const Engine&, const TelemetryAuditor&);
};
void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a) {
   if (e.temperature > a.alertThreshold)
      std::cout << "Warning: Engine overheat!\n";
   else
      std::cout << "Engine operating nominally at " << e.rpm << " RPM.\n";
}
int main() {
   Engine v8(105, 3200);
   TelemetryAuditor audit(100);
```

```
    inspectDiagnostics(v8, audit);
    return 0;
}
```

Compiler execution output:



Result: Compilation successful and program executed.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

These answers follow the scenarios and functional requirements in the supplied question bank. Pseudo-code is language-independent and is not presented as compiler-executable C++.

1. Corporate Vault Management with Static Data and Functions

```
CLASS BranchOffice
    PRIVATE:
        branchID : STRING
        branchName : STRING
        currentBalance : REAL
        STATIC totalVaultReserves : REAL = 0.0
        STATIC totalBranchesActive : INTEGER = 0

    PUBLIC:
        CONSTRUCTOR(id, name, openingDeposit)
            branchID = id
            branchName = name
            currentBalance = openingDeposit
            totalVaultReserves = totalVaultReserves + openingDeposit
            totalBranchesActive = totalBranchesActive + 1
        END CONSTRUCTOR

        METHOD Deposit(amount)
            IF amount > 0 THEN
                currentBalance = currentBalance + amount
                totalVaultReserves = totalVaultReserves + amount
            END IF
        END METHOD

        METHOD Withdraw(amount) RETURNS BOOLEAN
            IF amount > 0 AND currentBalance >= amount THEN
```

```
            currentBalance = currentBalance - amount
            totalVaultReserves = totalVaultReserves - amount
            RETURN TRUE
        END IF
        RETURN FALSE
    END METHOD

    STATIC METHOD GetVaultSummary()
        OUTPUT totalBranchesActive, totalVaultReserves
    END METHOD

    STATIC METHOD GetGlobalLiquidityRatio(targetReserve) RETURNS REAL
        IF targetReserve = 0 THEN RETURN 0
        RETURN totalVaultReserves / targetReserve
    END METHOD
END CLASS

MAIN
    CREATE Branch1("B01", "North", 200000)
    CREATE Branch2("B02", "South", 100000)
    Branch1.Deposit(150000)
    Branch2.Withdraw(50000)
    BranchOffice.GetVaultSummary()
    OUTPUT BranchOffice.GetGlobalLiquidityRatio(500000)
END MAIN
```

TRACE TABLE

Step	Branch 1	Branch 2	Total Reserves
Initial balances	200000	100000	300000
Branch 1 deposits 150000	350000	100000	450000
Branch 2 withdraws 50000	350000	50000	400000

2. Matrix Mathematical Engine with Friend Dot-Product

```
CLASS Matrix2D
    PRIVATE:
        grid : ARRAY[3][3] OF REAL
    PUBLIC:
        METHOD Initialize()
            FOR row = 0 TO 2
                FOR col = 0 TO 2
                    grid[row][col] = 0
                END FOR
            END FOR
        END METHOD
```

```
    METHOD SetElement(row, col, value)
      IF row >= 0 AND row < 3 AND col >= 0 AND col < 3
        grid[row][col] = value
      ELSE OUTPUT "Invalid matrix index"
      END IF
    END METHOD

    METHOD GetElement(row, col) RETURNS REAL
      IF index is valid THEN RETURN grid[row][col]
      ELSE OUTPUT "Invalid matrix index"; RETURN 0
      END IF
    END METHOD

    METHOD Display()
      OUTPUT grid as 3 rows and 3 columns
    END METHOD

    FRIEND FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B) RETURNS REAL
END CLASS

METHOD Matrix2D::Initialize()       // initialize every grid cell to zero
METHOD Matrix2D::SetElement(row,col,value) // validate and assign cell
METHOD Matrix2D::GetElement(row,col)       // validate and return cell
METHOD Matrix2D::Display()                 // print all nine values

FRIEND FUNCTION ComputeCrossTrace(A, B)
  total = 0
  FOR i = 0 TO 2
    total = total + A.grid[i][i] * B.grid[i][i]
  END FOR
  RETURN total
END FUNCTION

TRACE
i=0: 2 * 1 = 2; running total = 2
i=1: 4 * 3 = 12; running total = 14
i=2: 6 * 5 = 30; running total = 44
Final result = 44
```

3. Student Academic Roster Using Array of Objects

```
CLASS StudentRecord
  PRIVATE:
    rollNo : INTEGER
```

```
    studentName : STRING
    marks : ARRAY[3] OF REAL
    percentage : REAL
    grade : CHARACTER
  PUBLIC:
    CONSTRUCTOR()
      rollNo = 0; studentName = ""
      marks = [0, 0, 0]; percentage = 0; grade = 'F'
    END CONSTRUCTOR

    METHOD ReadData(r, name, m1, m2, m3)
      rollNo = r; studentName = name
      marks = [m1, m2, m3]
      CALL CalculateResult()
    END METHOD

  PRIVATE:
    METHOD CalculateResult()
      percentage = (marks[0] + marks[1] + marks[2]) / 3
      IF percentage >= 85 THEN grade = 'A'
      ELSE IF percentage >= 70 THEN grade = 'B'
      ELSE IF percentage >= 50 THEN grade = 'C'
      ELSE grade = 'F'
      END IF
    END METHOD

  PUBLIC:
    METHOD GetPercentage() RETURNS REAL
      RETURN percentage
    END METHOD
    METHOD Display()
      OUTPUT rollNo, studentName, percentage, grade
    END METHOD
END CLASS

MAIN
  DECLARE students[3] AS ARRAY OF StudentRecord
  students[0].ReadData(101, "Asha", 90, 88, 92)   // 90
  students[1].ReadData(102, "Bala", 75, 80, 77)   // 77.33
  students[2].ReadData(103, "Charan", 60, 65, 55) // 60

  FOR i = 0 TO 1
    FOR j = 0 TO 1-i
      IF students[j].GetPercentage() < students[j+1].GetPercentage()
```

```
            SWAP students[j], students[j+1]
          END IF
        END FOR
    END FOR
    OUTPUT "Rank list:"
    FOR each student IN students
        student.Display()
    END FOR
END MAIN

SORT TRACE
Initial order: Asha(90), Bala(77.33), Charan(60)
Pass 1: Asha > Bala; Bala > Charan; no swaps
Pass 2: Asha remains first; no swaps
Final order: Asha, Bala, Charan
```

4. Flexible Logger with Overloaded Functions and Inline Helpers

```
INLINE FUNCTION FormatPrefix(prefixTag) RETURNS STRING
    RETURN "[" + prefixTag + "] "
END FUNCTION

FUNCTION LogEvent(message : STRING)
    OUTPUT FormatPrefix("INFO") + message
END FUNCTION

FUNCTION LogEvent(message : STRING,
                  severityLevel : INTEGER = 1,
                  timestamp : STRING = "00:00:00")
    IF severityLevel = 1 THEN severity = "LOW"
    ELSE IF severityLevel = 2 THEN severity = "MEDIUM"
    ELSE IF severityLevel = 3 THEN severity = "CRITICAL"
    ELSE severity = "UNKNOWN"
    END IF
    OUTPUT "[" + severity + "] @ " + timestamp + " : " + message
END FUNCTION

FUNCTION LogEvent(errorCode : INTEGER, systemModule : STRING)
    hexCode = CONVERT errorCode TO HEXADECIMAL
    OUTPUT "[HARDWARE_FAULT] Module: " + systemModule +
           " -> Error Code: 0x" + hexCode
END FUNCTION

CLIENT CALLS
    LogEvent("System started")
```

```
    LogEvent("Temperature rising", 2)
    LogEvent(26, "SENSOR")

DEFAULT ARGUMENT TRACE
Call with message only selects the one-argument informational overload.
Call with message and severity uses timestamp default "00:00:00".
Call with integer and module selects the hardware-fault overload.
```

5. Hardware Register Bank with Return-by-Reference

```
CLASS RegisterBank
    PRIVATE:
        registers : ARRAY[8] OF INTEGER
        errorSink : INTEGER
    PUBLIC:
        CONSTRUCTOR()
            FOR i = 0 TO 7
                registers[i] = 0
            END FOR
            errorSink = -1
        END CONSTRUCTOR

        METHOD GetRegister(index) RETURNS REF INTEGER
            IF index >= 0 AND index < 8
                RETURN REFERENCE TO registers[index]
            ELSE
                OUTPUT "Alert: register index out of range"
                RETURN REFERENCE TO errorSink
            END IF
        END METHOD

        METHOD DisplayState()
            FOR i = 0 TO 7
                OUTPUT HEX(registers[i])
            END FOR
        END METHOD
END CLASS

MAIN
    CREATE bank AS RegisterBank
    bank.GetRegister(0) = 170
    bank.GetRegister(1) = 255
    readVal = bank.GetRegister(0)
    OUTPUT "Register 0:", HEX(readVal)
    bank.GetRegister(10) = 999
```

```
    bank.DisplayState()
END MAIN
```

EXPLANATION
GetRegister returns a reference to an existing array element, not a temporary local variable.
Therefore the returned reference can be used as the left side of assignment.
For an invalid index, the method returns the persistent errorSink reference so that
the invalid access does not write outside the register array.

Conclusion

This assignment covers function return semantics, static members, function overloading, arrays of objects, friend functions, class-based arrays, sorting, inline functions, default arguments, and reference-based register access. All five corrected Section A programs compiled and executed successfully using g++ with the C++17 standard.

Note: The included output images are terminal-style captures of the actual outputs produced by compiling and running the corrected programs. They are not screenshots from the student's personal compiler/IDE.